Private Evaluation - Final Private Leaderboard

IITM 6002W - Industrial Inventory Control RL Project

```
import numpy as np
from env import IndustrialInventoryEnv


# ============================================================
# PRIVATE LEADERBOARD
# DO NOT COMMIT THIS FILE TO THE PUBLIC REPOSITORY
# ============================================================

PROJECT_VERSION = "IITM-6002W-RL-Inventory-2026-v1"


PRIVATE_EPISODES = [
    ("stationary", 138947, [1.15, 0.92, 1.04], [80, 120, 100], [0.07, 0.02, 0.10]),
    ("seasonal",   624183, [0.93, 1.09, 0.86], [120, 100, 80], [0.02, 0.10, 0.04]),
    ("trend",      371429, [1.06, 0.85, 1.12], [100, 80, 120], [0.10, 0.03, 0.06]),
    ("shock",      905317, [0.85, 1.15, 0.94], [110, 120, 80], [0.05, 0.09, 0.01]),
    ("mixed",      247681, [1.10, 0.96, 1.15], [80, 100, 120], [0.08, 0.01, 0.07]),

    ("stationary", 753149, [0.90, 1.15, 1.06], [120, 80, 110], [0.01, 0.07, 0.10]),
    ("seasonal",   482357, [1.15, 1.00, 0.86], [100, 120, 80], [0.09, 0.04, 0.02]),
    ("trend",      196843, [0.86, 1.08, 1.15], [80, 110, 120], [0.04, 0.10, 0.01]),
    ("shock",      837461, [1.08, 0.85, 1.10], [120, 80, 100], [0.10, 0.02, 0.05]),
    ("mixed",      514729, [0.94, 1.15, 0.88], [100, 120, 90], [0.03, 0.08, 0.10]),

    ("stationary", 362971, [1.15, 0.86, 1.08], [90, 120, 80], [0.06, 0.10, 0.02]),
    ("seasonal",   781243, [0.87, 1.12, 1.15], [120, 80, 100], [0.10, 0.01, 0.08]),
    ("trend",      439817, [1.05, 1.15, 0.85], [80, 120, 110], [0.02, 0.09, 0.06]),
    ("shock",      653291, [1.15, 0.89, 0.97], [110, 80, 120], [0.09, 0.05, 0.01]),
    ("mixed",      218573, [0.88, 1.04, 1.15], [120, 100, 80], [0.01, 0.10, 0.07]),

    ("stationary", 947351, [1.03, 1.15, 0.87], [80, 110, 120], [0.08, 0.02, 0.10]),
    ("seasonal",   325619, [1.12, 0.86, 1.01], [120, 80, 110], [0.04, 0.10, 0.03]),
    ("trend",      569827, [0.85, 1.15, 1.09], [100, 120, 80], [0.10, 0.06, 0.01]),
    ("shock",      804263, [1.15, 0.95, 0.85], [80, 120, 100], [0.03, 0.09, 0.10]),
    ("mixed",      176549, [0.91, 1.15, 1.05], [120, 80, 110], [0.07, 0.01, 0.09]),
]


def _make_eval_config(
    episode_number,
    demand_profile,
    inventory_profile,
    delay_profile,
):
    return {
        "project_version": PROJECT_VERSION,
        "roll_number": "PRIVATE_EVAL",
        "variant_id": f"PRIVATE_E{episode_number:02d}",
        "demand_multiplier_profile": list(demand_profile),
        "initial_inventory_profile": list(inventory_profile),
        "lead_time_delay_profile": list(delay_profile),
    }


def _convert_policy_action(raw_action):
    action = np.asarray(raw_action)

    if action.shape != (3,):
        raise ValueError(
            "run_policy() must return exactly three quantities."
        )

    if not np.issubdtype(action.dtype, np.number):
        raise ValueError("Policy actions must be numeric.")

    if not np.all(np.isfinite(action)):
        raise ValueError("Policy actions must be finite.")

    if not np.all(action == np.round(action)):
        raise ValueError("Policy actions must be integers.")

    action = action.astype(np.int64)

    valid = (
        (action >= 0)
        & (action <= 100)
        & (action % 10 == 0)
    )

    if not np.all(valid):
        raise ValueError(
            "Actions must belong to "
            "{0,10,20,...,100}."
        )

    return IndustrialInventoryEnv.quantities_to_action_indices(
        action
    )


def run_private_evaluation_episodes(
    run_policy_fn,
    num_episodes=20,
):
```

```
if num_episodes != 20:
    raise ValueError(
        "Official private evaluation requires 20 episodes."
    )

total_cost = 0.0

for ep, spec in enumerate(
    PRIVATE_EPISODES,
    start=1,
):
    (
        scenario_mode,
        seed,
        demand_profile,
        inventory_profile,
        delay_profile,
    ) = spec

    config = _make_eval_config(
        ep,
        demand_profile,
        inventory_profile,
        delay_profile,
    )

    env = IndustrialInventoryEnv(
        student_config=config,
        scenario_mode=scenario_mode,
        domain_randomization=False,
    )

    obs, _ = env.reset(seed=seed)

    episode_cost = 0.0
    done = False

    while not done:
        raw_action = run_policy_fn(obs)

        action_indices = _convert_policy_action(
            raw_action
        )

        obs, reward, terminated, truncated, info = (
            env.step(action_indices)
        )

        episode_cost += float(
            info["costs"]["daily_total"]
        )

        done = terminated or truncated

    total_cost += episode_cost

return float(total_cost / len(PRIVATE_EPISODES))
```